

COMP 3311 数据库管理系统 — 课程总结 (Part 4: Lectures 16–24 — 存储、索引、查询处理、事务与恢复)

课程: COMP 3311 Database Management Systems

时间: 2026年7月12日–24日

覆盖: Lecture 16 – Lecture 24 (存储与文件结构、索引、查询处理、事务管理与恢复系统)

配套 Part 3: Lectures 12–15 (NoSQL & MongoDB)

参考符号约定:

B = 文件页数 nr = 元组数 M = 缓冲区页数
 bfr = 每页元组数 $V(A, r)$ = A 在 r 中不同值数
 HTi = B+-tree 索引高度 fi = 内部节点平均扇出

目录

1. Lecture 16: 存储与文件结构
2. Lecture 17: 索引 — 基础概念与有序索引
3. Lecture 18: 索引 — B+-树索引与哈希索引
4. Lecture 19: 查询处理 — 成本估算
5. Lecture 20: 查询处理 — 连接操作与表达式求值
6. Lecture 21: 查询处理 — 大小估算与查询优化
7. Lecture 22: 事务管理 — 事务与可串行化
8. Lecture 23: 事务管理 — 并发控制协议
9. Lecture 24: 事务管理 — 恢复系统与 NoSQL 事务

16. Lecture 16: 存储与文件结构

16.1 存储设备层次

访问速度 快 / 成本 高

└─ Cache (缓存)

└─ Main Memory / RAM (主存)

└─ SSD (固态硬盘)

└─ HDD (机械硬盘)

└─ 光盘 / 磁带

访问速度 慢 / 成本 低

← 主存储 (Primary), 易失

↕ I/O 边界

← 辅存储 (Secondary), 持久

← 第三存储 (Tertiary)

- 辅存储以页/块 (page/block) 为单位读写（通常 512B–16KB）
- 辅存储 I/O 远慢于主存操作 → DBMS 性能瓶颈

16.2 数据库缓冲区 (Database Buffer)

概念	说明
缓冲区 (Buffer)	主存中用于存储辅存储页面副本的区域
缓冲区管理器 (Buffer Manager)	操作系统子系统，管理缓冲空间
目标	最小化辅存储访问次数

请求页面的流程：

1. 若页已在缓冲区 → 直接返回地址
2. 若不在 → 分配空间（可能需要替换页面）→ 从辅存储读入

页面替换策略：

- LRU (Least Recently Used)：替换最久未使用的页
- MRU (Most Recently Used)：对嵌套循环等模式更好（如计算 join 时反复扫描同一关系）

16.3 记录组织

定长记录 (Fixed-Length Records)

组织方式	特点
相对位置	记录 i 从字节 $n \times (i-1)$ 开始；删除时移动记录（破坏指针）
空闲链表 (Free List)	删除时不移动记录，将第一个删除记录地址存在文件头中（空间高效）

- 文件阻塞因子： $bfr = \lfloor \text{页大小} / \text{记录大小} \rfloor$
- 所需页数： $\lceil \# \text{记录} / bfr \rceil$

变长记录 (Variable-Length Records)

方式	特点
字节串表示	顺序存储，附加结束标记；删除导致碎片
嵌入式标识	属性名前缀每个字段；额外空间但高效处理缺失字段
指针方法	锚页 + 溢出页，指针将相关记录链接
槽页结构 (Slotted-Page)	页头记录每个记录的位置和大小；可移动记录以保持连续 → 最常用

16.4 文件组织

组织方式	插入	删除	等值搜索	范围搜索	全扫描
堆文件 (Heap)	末尾追加 (2 I/O)	标记删除	0.5B (候选键) 或 B	B	B
顺序文件 (Sequential)	定位+溢出页 (B)	更新指针链 (B)	$\log_2 B$	$\log_2 B + \text{匹配页}$	B
哈希文件 (Hash)	哈希定位 (2 I/O)	哈希定位 (2 I/O)	1 I/O (无溢出)	B (不佳)	1.25B

堆文件：

- 适合全扫描和批量操作
- 等值搜索需全扫描

顺序文件：

- 记录按搜索键排序
- 适合范围查询和排序输出
- 需定期重组以维持顺序

哈希文件：

- 对等值搜索最优 (1 次 I/O)
- 对范围搜索最差

16.5 NoSQL 数据分布

分片 (Sharding)

- 将大数据集划分为更小块分配到不同服务器
- 模块哈希: $server = hash(key) \bmod n$
- 问题: 服务器数量变化时需要大量数据迁移

一致性哈希 (Consistent Hashing)

- 环形拓扑 $[0, 1]$, 服务器和键都映射到环上
- 键存储在其顺时针方向第一台服务器上
- 添加/移除服务器只需移动约 $k/(n+1)$ 个键

复制 (Replication)

- 每个物理服务器映射到环上多个位置 (虚拟节点/副本)
- 可扩展到多服务器冗余备份

17. Lecture 17: 索引 — 基础概念与有序索引

17.1 搜索键与索引

概念	说明
搜索键 (Search Key)	用于查找记录的属性 (集), 不一定是主键
索引文件 (Index File)	记录 $\langle \text{搜索键}, \text{指针} \rangle$ 的文件, 通常远小于数据文件
索引条目	通常 $\langle \text{搜索键值}, \text{该记录/页的指针} \rangle$

17.2 多级索引的效率

以 800 万条记录、每页 8 条记录、索引每页 100 条条目为例:

搜索方式	页访问成本
随机顺序 + 线性搜索	500,000 (平均)
按搜索键排序 + 二分搜索	20
一级索引 + 二分搜索	15
二级索引	9
三级索引 (根→叶→数据)	4

关键公式：树高 = $\lceil \log_{\text{fan-out}}(\text{叶子级别索引条目数}) \rceil$

17.3 有序索引 (Ordered Index)

每个节点中的索引条目按搜索键排序。搜索从根开始沿一条路径到达叶节点。

页 I/O 成本：树高（索引级别数） + 1（数据文件访问）

稠密 vs 稀疏索引

特性	稠密索引 (Dense)	稀疏索引 (Sparse)
索引条目	每个搜索键值一个条目	仅部分搜索键值有条目
空间开销	更大	更小
维护开销	更大	更小
搜索方式	直接找到记录	找到页面后顺序搜索

主索引 vs 辅助索引

特性	主索引 / 聚簇索引 (Primary/Clustering)	辅助索引 / 非聚簇索引 (Secondary/Non-Clustering)
数据文件排序	按搜索键排序	不按 搜索键排序
每个文件的索引数	最多 1 个	可以有多个
稠密/稀疏	通常是稀疏的	必须是稠密的 （为什么？数据不排序，无法通过页面指针定位）

索引顺序文件 (ISAM)：有序、顺序文件 + 主索引

17.4 非候选键搜索键上的索引

策略	方法	问题
策略 1	变长条目（一个键+多个指针）	实现复杂
策略 2	同一键多个条目	键冗余
策略 3（最常用）	额外间接层： 索引条目→指针页→记录	通常仅增加 1 次页访问

→ 也称为**倒排文件 (Inverted File / Postings List)**

17.5 复合搜索键 (Composite Search Key)

- 多属性搜索键：按**字典序**排列
- $(a1, a2) < (b1, b2)$ 当且仅当 $a1 < b1$ 或 $a1=b1$ 且 $a2 < b2$

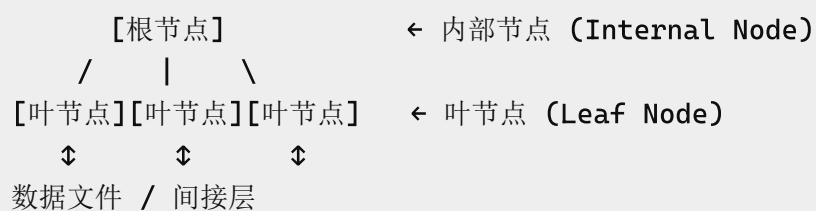
18. Lecture 18: 索引 — B+-树索引与哈希索引

18.1 B+-树 vs 有序索引

特性	有序索引 (ISAM)	B+-树
文件增长	性能退化（溢出页）	自动局部重组
重组	需定期全文件重组	不需要
平衡性	不保证	始终平衡 （根→叶等长）
额外开销	低	插入/删除/空间开销

→ B+-树广泛用于所有商业 DBMS

18.2 B+-树结构与性质



节点容量（扇出 = n）：

节点类型	最少指针	最多指针	最少值	最多值
内部节点	$\lceil n/2 \rceil$	n	$\lceil n/2 \rceil - 1$	n-1
叶节点	$\lceil (n-1)/2 \rceil + 1$	n	$\lceil (n-1)/2 \rceil$	n-1
根（非叶）	2	n	1	n-1
根（叶）	0	n	0	n-1

B+-树阶 (Order): $\lceil (n-1)/2 \rceil$ = 叶节点中的最小值数

内部节点指针规则

$P_1 \ K_1 \ P_2 \ K_2 \ \dots \ P_{n-1} \ K_{n-1} \ P_n$

- P_1 指向 $< K_1$ 的子树
- P_i 指向 $\geq K_{i-1}$ 且 $< K_i$ 的子树
- P_n 指向 $\geq K_{n-1}$ 的子树

叶节点

- 包含实际搜索键值 + 记录指针（或间接层指针）
- 最后指针 P_n 指向**右兄弟节点**（支持范围扫描）

18.3 B+-树四种类型

类型	数据文件排序	键类型	索引密度
聚簇、候选键（记录指针）	按搜索键	候选键	稠密
聚簇、候选键（页面指针）	按搜索键	候选键	稀疏
聚簇、非候选键	按搜索键	非候选键	稠密
非聚簇、候选键	不排序	候选键	稠密
非聚簇、非候选键	不排序	非候选键	稠密（+ 间接层）

18.4 B+-树查询

1. 从根开始
2. 找最小的 $K_i > v$, 沿 P_i （左指针）向下
若无 $K_i > v$, 沿 P_n （最后指针）向下
3. 重复直到叶节点
4. 在叶中找 $K_i = v$:
 - 找到 → 沿 P_i 到记录
 - 未找到 → 不存在

- 路径长度: $\leq \lceil \log_{\lceil n/2 \rceil}(K) \rceil$ (K = 搜索键值数)
- 典型 $n \approx 100$, 100 万搜索键值时最多 4 个节点

18.5 B+-树更新

插入

1. 找到应插入的叶节点 L
2. 若 L 有空间 → 插入
3. 若 L **溢出** → 分裂 (Split)
 - **Copy Up**: (叶节点) 前 $\lfloor n/2 \rfloor$ 值留在 L, 复制 $\lfloor n/2 \rfloor + 1$ 值并插入父节点
 - **Push Up**: (内部节点) $\lfloor n/2 \rfloor$ 位置的值被推到父节点
 - 分裂可递归到根 → 树增高

删除

1. 找到值所在叶节点 L, 删除
2. 若 L **欠满** ($< \lfloor (n-1)/2 \rfloor$):
 - 先尝试从兄弟**重新分配** (借值)
 - 重新分配失败 → **合并** (Merge)
 - 合并可传播到根 → 树变矮

18.6 B+-树批量加载 (Bulk Loading)

- 比逐条插入高效得多
- 步骤: 排序数据条目 → 按最小占用装载叶节点 → 自底向上构建内部节点
- 每个叶节点只写一次

18.7 哈希索引

特性	说明
组织方式	索引文件 (非数据文件) 是哈希文件
索引类型	始终是辅助索引
优点	等值搜索极快 (1 I/O)
缺点	不支持范围搜索

静态哈希缺陷:

- 页数固定 → 数据库增长导致溢出页过多 → 性能退化
- 需要动态哈希技术 (可动态修改哈希函数)

18.8 索引选择考量

因素	B+-树	哈希索引
等值搜索	✓ 高效	✓ 最高效
范围搜索	✓ 高效	X 不支持
使用频率	~90% 查询	主要用于主存哈希
更新开销	中等	低

19. Lecture 19: 查询处理 — 成本估算

19.1 查询处理概览

SQL 查询 → 解析器 (Parser) → 关系代数表达式

↓

查询结果 ← 求值引擎 (Evaluation) ← 优化器 (Optimizer) → 执行计划

- 同一查询可能有**多个等价的关系代数表达式**
- 每个操作可用**不同算法**实现
- 优化器选择**预估成本最低**的执行计划

19.2 成本度量

- 以**页 I/O 次数**度量成本
- 忽略 CPU 成本和顺序/随机 I/O 差异 (简化假设)
- **不包含**最终输出写入辅存储的成本
- 使用数据库目录中的统计信息

19.3 选择操作 — 等值搜索

算法	适用条件	候选键成本	非候选键成本
A1: 线性搜索	总是可用	$B/2$	B
A2: 二分搜索	文件按搜索键排序	$\lceil \log_2 B \rceil$	$\lceil \log_2 B \rceil + \text{匹配页数}$
A3: 聚簇索引	B+-tree 聚簇索引	$HT_i + 1$	$HT_i + \text{匹配页数}$
A4: 非聚簇索引	B+-tree 辅助索引	$HT_i + 1$	$HT_i + \text{间接指针} + \text{记录数}$

哈希索引（辅助索引，候选键）： $1 + 1$ （或 $1.2 + 1$ 若有溢出）

19.4 选择操作 — 范围搜索

算法	条件
A5: 聚簇 B+-tree	$\sigma_{A \geq v}$: 用索引找首记录，顺序扫描； $\sigma_{A \leq v}$: 不用索引，顺序扫描到 $A > v$
A6: 非聚簇 B+-tree	$\sigma_{A \geq v}$: 扫描索引叶页找指针； $\sigma_{A \leq v}$: 扫描索引叶页直到 $A > v$

19.5 选择操作 — 复杂谓词

算法	谓词类型	方法
A7	合取 (AND) 单索引	选成本最低的 q_i 用索引，在缓冲中检查其余条件
A8	合取 复合索引	使用复合（多属性）索引
A9	合取 指针求交	多个索引扫描 + 指针交集 + 取记录
A10	析取 (OR) 指针求并	所有条件都需索引；指针并集 + 取记录

否定 (NOT): 使用线性搜索；若否定条件选择性很高且有索引，可间接使用索引。

19.6 外部排序 (External Sort-Merge)

当数据太大无法全部装入主存时使用。

算法:

1. **Pass 0 (创建排好序的 Run)**: 每 M 页读入缓冲，排序，写回 $\rightarrow$ 创建 B/M 个 run
2. **合并 Pass**: 合并 $M-1$ 个 run，直到所有 run 合并为一个

成本:

- Pass 数: $1 + \lceil \log_{M-1}(B/M) \rceil$
- I/O 成本: $2B \times (1 + \lceil \log_{M-1}(B/M) \rceil)$

两次完成排序所需缓冲页数: $M \approx \sqrt{B}$

20. Lecture 20: 查询处理 — 连接操作与表达式求值

20.1 投影操作

场景	方法
无重复消除	生成结果元组时移除不需要的属性 → 无额外成本
需要 DISTINCT	修改的外部排序（排序时移除不需要属性 + 合并时消除重复）
索引可用	若稠密索引包含所有需要的属性 → 仅索引扫描

20.2 连接操作 (JOIN)

参考符号：r（外关系），s（内关系）， n_r （r 的元组数）， B_r （r 的页数），M（缓冲页数）

块嵌套循环连接 (Block Nested-Loop Join)

条件	成本公式
最坏情况	$B_r \times B_s + B_r$
最好情况	$B_r + B_s$
优化（M-2 页阻塞）	$\lceil B_r / (M-2) \rceil \times B_s + B_r$

- 若某关系能放入缓冲 → 用它作**内关系 s**
- 若都不能放入 → 用**较小关系作外关系 r**
- 若连接属性是内关系的主键 → 可在**第一个匹配后停止**内循环

索引嵌套循环连接 (Indexed Nested-Loop Join)

成本： $B_r + n_r \times c$

- c = 索引搜索 + 取匹配 s 元组的成本
- **最佳场景：** 外关系选择性高（少量元组满足条件）
- 若两边都有索引 → 用较小的关系作外关系

归并连接 (Merge-Join)

成本： $B_r + B_s$ （已排序）或 $B_r + B_s + \text{排序成本}$ （未排序）

- 仅适用于等值连接和自然连接
- 每页只读一次（假设同值元组都在缓冲中）

哈希连接 (Hash-Join)

成本： $3 \times (B_r + B_s)$

阶段	I/O
创建分区	1 读 + 1 写（两个关系各一次）
计算连接	1 读（两个关系各一次）

算法：

1. 用哈希函数 h 将 r 和 s 分区为 n 个桶
 - 只比较分区 r_i 和 s_i （同哈希值的元组）
2. 对每个 i ：将 r_i 载入缓冲 → 建主存哈希索引 → 逐页探测 s_i

约束：

- n 个分区：每个 build input 分区必须 $\leq M$ 页
- $M \geq n+1$ （每个分区一个缓冲页 + 一个输入页）
- build input 应选**较小关系**

20.3 复杂连接

条件	方法
合取条件	计算最选择性连接的 join，在缓冲中检查其余条件
析取条件	各简单连接的 join 结果取并集（仅在所有条件都选择性高时有用）

20.4 表达式求值

物化 (Materialization)

- 一次求值一个操作，结果写入临时关系
- **总是可用**，但中间结果的读写成本高

流水线 (Pipelining)

- 多个操作同时求值，结果直接传递
 - **成本远低于物化**（无需写入辅存储）
 - **需求驱动（惰性）**：顶层请求元组，逐层下推
 - **生产者驱动（急切）**：持续产生元组，放入缓冲
 - **阻塞操作**：归并连接、哈希连接（需等全部输入才能输出）
-

21. Lecture 21: 查询处理 — 大小估算与查询优化

21.1 数据库统计信息

统计量	含义
n_r	r 的元组数
B_r	r 的页数
l_r	r 的元组大小（字节）
bfr	r 的阻塞因子
$V(A, r)$	A 在 r 中不同值数量
HT_i	索引 i 的高度
LB_i	索引 i 底层页数

21.2 选择基数与选择性

概念	公式
选择基数 $SC(q, r)$	满足谓词 q 的平均元组数
选择性 $Selectivity(q, r)$	$SC(q, r) / n_r$ (0 到 1 之间)

等值选择（非候选键）： $SC(A=v, r) = n_r / V(A, r)$

等值选择（候选键）： $SC(A=v, r) = 1$

范围选择： $SC(A < v, r) = n_r \times (v - \min(A, r)) / (\max(A, r) - \min(A, r))$

21.3 直方图 (Histogram)

- 解决**均匀分布假设**不准确的问题

- **等宽直方图**：按值的范围等分
- **等深直方图**：按元组数等分

21.4 复杂选择的大小估算

合取： $SC(\sigma_{q_1 \wedge q_2}, r) = n_r \times (s_1 \times s_2)$ (假设属性独立)

析取： $SC(\sigma_{q_1 \vee q_2}, r) = n_r \times (1 - (1-s_1)(1-s_2))$

21.5 连接大小估算

条件	估算结果大小
$r \cap s = \emptyset$	$n_r \times n_s$ (笛卡尔积)
$r \cap s$ 是 r 的候选键	$\leq n_s$
$r \cap s$ 是 s 中引用 r 的外键	$= n_s$
$r \cap s$ 不是任一方键	取 $\min(n_r \times n_s / V(A, s), n_s \times n_r / V(A, r))$

21.6 其他操作大小估算

操作	估算大小
投影	$V(A, r)$
分组	$V(A, r)$
并	$\text{size}(r) + \text{size}(s)$
交	$\min(\text{size}(r), \text{size}(s))$
差	$\text{size}(r)$

21.7 查询优化

基于成本的优化 (Cost-Based)

1. 使用等价规则生成逻辑等价求值计划
2. 估算每个计划的成本
3. 执行成本最小的计划

启发式优化 (Heuristic)

1. 尽早执行选择 (减少元组数)
2. 尽早执行投影 (减少元组大小)
3. 利用现有索引
4. 先执行最具选择性的操作

21.8 关系代数等价规则

规则	说明
选择级联	$\sigma_{C_1 \wedge C_2}(R) \equiv \sigma_{C_1}(\sigma_{C_2}(R))$
选择交换	$\sigma_{C_1}(\sigma_{C_2}(R)) \equiv \sigma_{C_2}(\sigma_{C_1}(R))$
投影级联	$\pi_{a_1}(R) \equiv \pi_{a_1}(\pi_{a_2}(R)) \quad (a_1 \subseteq a_2)$
连接交换	$R \bowtie S \equiv S \bowtie R$
连接结合	$(R \bowtie S) \bowtie T \equiv R \bowtie (S \bowtie T)$
选择+连接交换	$\sigma_c(R \bowtie S) \equiv \sigma_c(R) \bowtie S \quad (c \text{ 仅涉及 } R)$
投影+连接分配	$\pi_a(R \bowtie S) \equiv \pi_a(\pi_{a_1}(R) \bowtie \pi_{a_2}(S))$

22. Lecture 22: 事务管理 — 事务与可串行化

22.1 事务与 ACID 属性

属性	含义	处理机制
Atomicity (原子性)	全部执行 或 全部不执行	恢复系统
Consistency (一致性)	事务隔离执行保持数据库一致	并发控制
Isolation (隔离性)	并发事务互相不可见中间结果	并发控制
Durability (持久性)	提交后的更改不丢失	恢复系统

22.2 事务状态转换

Active → Partially Committed → Committed
 ↓
 Failed → Aborted (→ 可选择重启或终止)

22.3 调度的正确性

调度类型	说明
串行调度 (Serial)	一次只执行一个事务（无并发）
可串行化调度 (Serializable)	并发执行效果等价于某串行调度
冲突可串行化	可通过交换非冲突指令变成串行调度

22.4 冲突 (Conflict)

操作组合	是否冲突
read(Q) + read(Q)	不冲突
read(Q) + write(Q)	冲突
write(Q) + read(Q)	冲突
write(Q) + write(Q)	冲突

→ 如果两个指令连续且不冲突，**交换它们不改变结果**

22.5 优先图 (Precedence Graph)

- 顶点 = 事务
- 边 $T_i \rightarrow T_j = T_i$ 比 T_j 先访问冲突的数据项
- **调度冲突可串行化 $\Leftrightarrow$ 优先图无循环**

22.6 可恢复性 (Recoverability)

调度类型	条件
可恢复 (Recoverable)	T_j 读取 T_i 写的数据后， T_j 在 T_i 之后提交
无级联回滚 (Cascadeless)	T_j 只能在 T_i 提交后 才能读取 T_i 写的数据
关系	级联无级联 $\subset$ 可恢复

→ 调度必须**可恢复**，且最好**级联无级联**

23. Lecture 23: 事务管理 — 并发控制协议

23.1 锁 (Lock)

锁模式	可执行操作	请求指令
共享锁 (Shared/S)	只能读	lock-s(Q)
排他锁 (Exclusive/X)	可读可写	lock-x(Q)
解锁	释放锁	unlock(Q)

锁兼容矩阵

持有\请求	S	X
S	✓ Granted	X Denied
X	X Denied	X Denied

→ 任意数量的事务可共享 S 锁；若任何事务持有 X 锁，其他事务都不能获锁。

23.2 两阶段锁协议 (Two-Phase Locking, 2PL)

阶段	允许操作
Growing Phase (增长阶段)	获取或升级锁
Shrinking Phase (收缩阶段)	释放或降级锁

- 2PL → 调度必为**冲突可串行化**
- 事务可按**锁点（获得最后锁的时刻）**排序
- ⚠️ 不是所有冲突可串行化的调度都被 2PL 允许

23.3 2PL 可恢复性改进

变体	规则	保证
严格 2PL (Strict 2PL)	所有 排他锁 保持到事务提交	级联无级联 + 可恢复
严格 2PL (Rigorous 2PL)	所有锁 保持到事务提交	级联无级联 + 可恢复

23.4 死锁 (Deadlock)

两个事务互相等待对方释放锁 → 必须回滚其中一个。

死锁处理

方法	策略
死锁预防	
— 排序锁请求	所有事务按相同（部分/全）序申请锁
— Wait-Die	老事务等待年轻的；年轻事务回滚 ("die")
— Wound-Wait	年轻事务等待老的；老事务"伤害"（回滚）年轻事务
死锁检测	使用 等待图 (Wait-for Graph) ；有循环 = 死锁
死锁恢复	选择牺牲品（最小成本）→ 事务回滚

等待图：顶点 = 事务，边 $T_i \rightarrow T_j = T_i$ 等待 T_j 释放数据项。

23.5 锁管理器实现

- 锁管理器作为独立进程运行
- 维护**锁表**（主存哈希表，按数据项索引）
- 锁请求按 FIFO 顺序排队和授权
- 解锁导致检查队列中下一个请求是否可授权

23.6 时间戳协议 (Timestamp-Ordering Protocol)

每个事务 T_i 在开始前获得固定时间戳 $TS(T_i)$ 。时间戳决定可串行化顺序。

每个数据项 Q 维护：

- **RTS(Q)**：成功执行 $read(Q)$ 的最大时间戳
- **WTS(Q)**：成功执行 $write(Q)$ 的最大时间戳

读操作规则

条件	动作
$TS(T_i) < WTS(Q)$	回滚 T_i （试图读已被新事务覆盖的值）
$TS(T_i) \geq WTS(Q)$	执行读， $RTS(Q) = \max(RTS(Q), TS(T_i))$

写操作规则

条件	动作
$TS(T_i) < RTS(Q)$	回滚 T_i (新事务已读旧值)
$TS(T_i) < WTS(Q)$	回滚 T_i (新事务已写新值)
否则	执行写, $WTS(Q) = TS(T_i)$

Thomas 写规则 (Thomas' Write Rule)

原规则	Thomas 规则
$TS(T_i) < WTS(Q) \rightarrow$ 回滚	$TS(T_i) < WTS(Q) \rightarrow$ 忽略此写

→ 如果新事务已写了更新的值, 忽略旧值写入 (更高效, 避免不必要回滚)

23.7 多版本并发控制 (Multiversion)

- 保持数据项的多个版本 (标记时间戳)
- 每个版本 Q_k 包含: Content, $WTS(Q_k)$, $RTS(Q_k)$

多版本时间戳排序读:

- 读总是成功 → 返回 $WTS \leq TS(T_i)$ 的最大版本的内容
- 读**从不失败、从不等待**

多版本时间戳排序写:

- $TS(T_i) < RTS(Q_k) \rightarrow$ 回滚
- $TS(T_i) = WTS(Q_k) \rightarrow$ 覆盖内容
- $TS(T_i) > WTS(Q_k) \rightarrow$ 创建新版本

24. Lecture 24: 事务管理 — 恢复系统与 NoSQL 事务

24.1 DBMS 故障分类

故障类型	说明
事务故障 (Transaction Failure)	
— 逻辑错误	事务因内部错误无法完成（如逻辑错误、无效输入）
— 系统错误	DBMS 必须终止活动事务（如死锁）
系统崩溃 (System Crash)	因电源、硬件或软件故障导致；假设非易失性存储内容不被破坏（Fail-stop 假设）
辅存储故障 (Secondary Storage Failure)	磁盘故障（如磁头碰撞）破坏部分或全部磁盘存储；假设可被检测（校验和）

24.2 恢复算法

恢复算法有两个部分：

- 1. 正常事务处理期间采取的操作 — 确保有足够信息用于恢复
- 2. 故障发生后的操作 — 将数据库恢复到确保原子性、持久性和一致性的状态

目标：尽管发生故障，仍确保事务原子性、持久性和数据库一致性。

24.3 数据访问假设

```
x1, y1  x, y
Buffer Pages (缓冲页)
    buffer page A ← input(A)
    buffer page B → output(B)
    read(x), write(y)
Private work area (T1), Private work area (T2)
Main Memory ↔ Secondary Storage (Physical pages)
```

概念	说明
物理页 (Physical Pages)	驻留在辅存储上的数据库页面
缓冲页 (Buffer Pages)	临时驻留在主存中的数据库页面
input(B)	将物理页 B 从辅存储传入缓冲区
output(B)	将缓冲页 B 替换到辅存储
read(x)	将数据库项 x 的值赋给局部变量 x _i （可能需要 input）
write(x)	将局部变量 x _i 的值赋给缓冲区中的 x（不需要立即 output）

- 每个事务有**私有工作区**（局部副本）
- 事务在**首次访问** x 时执行 $\text{read}(x)$ ，后续访问操作局部副本 x_i
- 在**最后一次访问** x_i 后执行 $\text{write}(x)$
- $\text{output}(BX)$ **不需要**立即跟随 $\text{write}(x)$

24.4 恢复与原子性

问题：修改数据库但不确保事务提交，可能使数据库处于不一致状态。故障可能发生在部分 output 操作之后。

解决方案：首先向稳定存储输出**描述修改的信息**（日志），而不直接修改数据库本身。

24.5 基于日志的恢复 (Log-Based Recovery)

日志 (Log)：记录所有数据库更新活动的记录序列，保存在**稳定存储**（如磁盘）上。

日志记录	写入时机	内容
$\langle T_i \text{ start} \rangle$	事务 T_i 开始时	注册事务
$\langle T_i, x, v_1, v_2 \rangle$	T_i 执行 $\text{write}(x)$ 之前	v_1 =旧值, v_2 =新值
$\langle T_i \text{ commit} \rangle$	T_i 完成最后一条指令时	标记提交

延迟数据库修改 (Deferred Database Modification)

所有修改**记录到日志**，但写入数据库**推迟到部分提交后**。

- 日志记录仅含**新值 v** ： $\langle T_i, x, v \rangle$
- 当 T_i 部分提交时，写入 $\langle T_i \text{ commit} \rangle$
- 最后读取日志记录并执行被推迟的写入

恢复规则：事务需要 **redo** $\Leftrightarrow$ 日志中同时有 $\langle T_i \text{ start} \rangle$ 和 $\langle T_i \text{ commit} \rangle$

- $\text{redo}(T_i)$ ：将所有被 T_i 更新的数据库项设为其新值
- 既无 start 也无 $\text{commit} \rightarrow$ **忽略**，重启事务

立即数据库修改 (Immediate Database Modification)

未提交事务的数据库更新**可以在 write 发出时立即执行**。

- 日志必须同时包含**旧值 v₁** 和**新值 v₂**: `<Ti, x, v1, v2>`
- 更新日志记录必须在数据库项被写入**之前**写入
- 更新页的 output 可在事务提交**之前或之后**进行
- output 的顺序可能与写入缓冲区的顺序**不同**

恢复规则:

- **undo(T_i)**: 从最后一条记录向后, 将 T_i 更新过的所有项恢复为**旧值**
- **redo(T_i)**: 从第一条记录向前, 将 T_i 更新过的所有项设为**新值**
- 两种操作必须是**幂等的 (idempotent)**
- T_i 需要 undo ⇔ 日志中有 `<Ti start>` 但无 `<Ti commit>`
- T_i 需要 redo ⇔ 日志中同时有 `<Ti start>` 和 `<Ti commit>`
- **undo 先执行, redo 后执行**

延迟 vs 立即修改对比

特性	延迟修改	立即修改
日志记录的旧值	不需要	需要 (用于 undo)
数据库写入时机	提交后	write 发出时
恢复操作	仅 redo	undo + redo
未提交事务处理	忽略 (重启事务)	undo 恢复旧值

24.6 检查点 (Checkpoints)

问题:

1. 搜索整个日志文件**耗时**
2. 可能**不必要地 redo** 已输出更新的事务

检查点操作:

1. 输出缓冲中所有日志记录到稳定存储
2. 输出所有已修改的缓冲页到辅存储
3. 写入 `<checkpoint>` 日志记录

事务在检查点执行期间**不能执行任何更新操作**。

串行执行下的恢复（含检查点）

1. 从日志末尾**向后扫描**找到最近的 `<checkpoint>`
2. 继续向后扫描直到找到 `<Ti start>` 记录
3. 只需考虑此 start 记录**之后**的日志部分；之前的部分可忽略
4. **向前扫描**日志（从 T_i 开始）：
 - 无 `<Ti commit>` → 执行 `undo(Ti)`（仅在立即修改时）
 - 有 `<Ti commit>` → 执行 `redo(Ti)`

示例：



24.7 并发事务的恢复

当允许多事务并发执行时（使用 **strict 2PL**）：

- 所有事务共享**单一缓冲区和单一日志**
- 不同事务的日志记录可能在日志中**交错**
- 一个缓冲页可能包含**多个事务**更新的数据库项

检查点格式变更

`<checkpoint L>` — L 是检查点时**活跃事务**（未提交）的列表。

并发恢复算法

第一阶段 — 构建列表：

1. 初始化 `undo-list` 和 `redo-list` 为空
2. 从日志末尾**向后扫描**，到第一个 `<checkpoint L>` 记录停止
3. 对每个记录：
 - 若是 `<Ti commit>` → T_i 加入 **redo-list**
 - 若是 `<Ti start>` 且 T_i 不在 `redo-list` → T_i 加入 **undo-list**
4. 对 L 中每个 T_i ，若不在 `redo-list` → 加入 **undo-list**

第二阶段 — 执行恢复：

- 1. 向后扫描日志（从最后记录到所有 undo-list 事务的 start）→ 执行 **undo**
- 2. 向前扫描日志（从 checkpoint 到末尾）→ 对 redo-list 执行 **redo**

| 向后 undo 恢复原始值；向前 redo 设置每项为最新值。

24.8 写前日志 (Write-Ahead Logging, WAL)

若日志记录**缓冲在主存中**（非直接输出），须遵守 WAL 规则：

规则	说明
1. 顺序输出	日志记录按 创建顺序 输出到稳定存储
2. 提交前输出	T _i 进入提交状态仅在 <code><T_i commit></code> 输出到稳定存储 之后
3. 数据输出前先输出日志	缓冲页输出到数据库 之前 ，该页相关的所有日志记录必须已输出

| **Log Force**：通过强制将所有日志记录（含 commit 记录）输出到稳定存储来提交事务。

24.9 NoSQL 分布式事务

本地 vs 全局事务

类型	说明
本地事务 (Local)	仅访问和更新 一个节点 上的数据
全局事务 (Global)	访问和更新 多个节点 上的数据

事务协调架构

角色	职责
事务协调器 (Transaction Coordinator)	启动事务、分发子事务到适当站点、协调终止
本地事务管理器 (Local Transaction Manager)	维护日志用于恢复、协调该站点事务的提交/中止

| **提交协议**：确保原子性 — 多站点事务必须在所有站点都提交或都中止。

24.10 CAP 定理

分布式数据库系统**不能同时保证**以下三个属性：

属性	含义
Consistency（一致性）	所有节点同时看到相同数据
Availability（可用性）	每个请求都能收到成功/失败的响应
Partition Tolerance（分区容忍）	即使部分节点宕机，系统继续工作

在网络分区存在时，必须在**一致性**和**可用性**之间选择：

- **关系型 DBMS** → 选择一致性（一致性优先）
- **NoSQL DBMS** → 选择可用性（可用性优先）

24.11 BASE 原则（最终一致性）

NoSQL DBMS 遵循 **BASE** 原则而非 ACID：

属性	说明
Basically Available（基本可用）	遵守 CAP 定理的可用性保证
Soft state（软状态）	系统可随时间变化，即使没有输入（因网络延迟更新）
Eventually consistent（最终一致）	系统随时间最终将变得一致

24.12 事务管理总结

方面	关系型 DBMS（ACID）	NoSQL DBMS（BASE）
核心保证	强一致性	最终一致性
并发控制	锁协议 (2PL) / 时间戳协议	更宽松
恢复机制	日志 + WAL + 检查点	复制 + 最终一致
分布式事务	提交协议（原子跨站点）	CAP 保证：可用性 > 一致性
适用场景	银行、金融	社交网络、Web 应用

存储与文件结构 (L16)

- 存储层次：主存(易失) → SSD/HDD(持久) → 磁带(归档)
- 数据库缓冲：缓冲区管理器, LRU/MRU 替换策略
- 记录组织：定长(空闲链表) vs 变长(槽页结构最常用)
- 文件组织对比：堆(全扫描) vs 顺序(范围查询) vs 哈希(等值查询)
- NoSQL 分布：分片, 一致性哈希(环), 复制

索引基础 (L17)

- 搜索键 ≠ 主键(可为任意属性)
- 多级索引：树高 = $\log(\text{扇出})(\text{条目数})$
- 稠密 vs 稀疏：每个值 vs 部分值
- 主/聚簇索引 vs 辅助/非聚簇索引：排序 vs 不排序
- 非候选键索引：间接层/倒排文件
- 复合搜索键：字典序排列

B+-树索引 (L18)

- 结构：平衡树, 根→叶等长
- 节点：内部节点($\lceil n/2 \rceil \sim n$ 指针) + 叶节点
- 查询：对数复杂度($\sim \log_{\text{扇出}}(K)$)
- 更新：溢出→分裂(Copy/Push Up); 欠满→重新分配/合并
- 批量加载：排序+自底向上(效率高)
- 四种形态：聚簇/非聚簇 × 候选键/非候选键
- 哈希索引：等值快, 不支持范围, 静态有缺陷

查询处理 (L19-L21)

- 成本度量：页I/O, 忽略CPU/顺序vs随机
- 选择算法：线性/二分/聚簇索引/辅助索引
- 外部排序：Pass数 = $1 + \lceil \log_{M-1}(B/M) \rceil$, 两遍需 $M \approx \sqrt{B}$
- 连接算法：块嵌套循环/索引嵌套循环/归并/哈希连接
- 表达式求值：物化 vs 流水线
- 大小估算：基数SC, 选择性selectivity, 直方图
- 等价规则：选择级联/交换, 连接交换/结合, 选择和投影下推
- 优化：成本-based + 启发式(早选择/早投影/用索引)

事务管理：并发控制 (L22-L23)

- ACID：原子性(恢复) + 一致性(并发控制) + 隔离性(并发控制) + 持久性(恢复)
- 调度：串行 → 可串行化 → 冲突可串行化(优先图无环)
- 可恢复性：可恢复 → 级联无级联
- 锁：共享(S)/排他(X), 锁兼容矩阵
- 2PL：Growing→Shrinking, 保证冲突可串行化
- 严格2PL：排他锁(严格) 或 所有锁(Rigorous) 保持到提交
- 死锁：预防(排序/Wait-Die/Wound-Wait) + 检测(等待图)
- 时间戳协议：RTS/WTS 规则 + Thomas写规则
- 多版本：保留旧版本, 读永不失败, 版本按时间戳管理

事务管理：恢复系统 (L24)

- 故障分类：事务故障(逻辑/系统) + 系统崩溃 + 辅存储故障
- 数据访问：read(x)/write(x), input/output, 私有工作区, 缓冲页
- 日志基础：<Ti start> + <Ti, x, v1, v2> + <Ti commit>

- └─ 延迟修改：仅存储新值，提交后写入，恢复仅做 redo
- └─ 立即修改：存储新旧值，立即写入，恢复做 undo + redo
- └─ 检查点：输出日志+缓冲页 → <checkpoint>，限制扫描范围
- └─ 并发恢复：<checkpoint L> 含活跃事务列表，undo-list + redo-list
- └─ WAL规则：日志顺序输出，commit先于数据，数据输出前日志已输出
- └─ NoSQL事务：本地(单节点) vs 全局(多节点)，协调器+本地管理器
- └─ CAP定理：一致性/可用性/分区容忍 三者不可兼得
- └─ BASE：基本可用 + 软状态 + 最终一致性 (vs ACID)

课程参考："Fundamentals of Database Systems"（教材章节标注于各 Lecture 幻灯中）

关联文件：

- [COMP3311_Lecture_Summary_Part1](#) — E-R 模型、关系设计、范式化、关系代数
- [COMP3311_Lecture_Summary_Part2](#) — SQL DML & DDL
- [COMP3311_Lecture_Summary_Part3](#) — NoSQL & MongoDB
- **Part 4 完结篇** — 覆盖 L16–L24，至此 COMP 3311 全部课程笔记完成。